BDA Week 4-5 — Finding Similar Items: Shingling, MinHash, LSH

Module 2 of Phase-1 Final Prep · Heaviest numerical-trace topic in the course

BDA Final Exam Prep · Comprehensive Tutor

Week 04 – 05 · Finding Similar Items (Shingling → MinHash → LSH)

- **§1 — Beginning Key Notes (Study Compass)**
- **§2 — The Near-Duplicate Problem: Why Pairwise Comparison Doesn't Scale**
- **§3 — Step 1: Shingling (Documents → Sets)**
  - **Definition — k-shingle (k-gram)**
  - **Choosing k**
  - **Hashing Shingles to Compress**
  - **Sets → Boolean Matrix**
- **§4 — Jaccard Similarity (The Right Baseline for Sets)**
  - **Properties**
  - **Slide Example — three documents**
  - **Why Jaccard?**
  - **Reading Jaccard Off the Boolean Matrix**
- **§5 — Step 2: MinHashing (Sets → Short Signatures)**
  - **The MinHash Idea**
  - **The MinHash Theorem (the killer lemma)**
  - **Why It Works in Practice (Estimation)**
  - **Slide Example (Worked) — 7 rows × 4 columns, 3 permutations**
- **§6 — Implementing MinHash Without Permutations**
  - **The Hash Family**
  - **The Algorithm (slide form, memorize this)**
  - **Step-by-Step Trace (slide example: 5 rows, 2 columns, 2 hashes)**
  - **Why** hih_i ≈ **random permutation**
- **§7 — Step 3: Locality-Sensitive Hashing (LSH)**
  - **Banding Construction**
  - **Visual**
  - **Why It Works**
  - **Probability Calculation**
  - **Numerical Verification (slide example:** b=20b = 20**,** r=5r = 5**)**
  - **What Bands Detect**

Why “Locality Sensitive”

§8 — Tuning b and r: The S-Curve Threshold

The Threshold Formula

Worked: b=20b = 20, r=5r = 5, n=100n = 100

Trade-off

§9 — LSH for Other Distances (briefly)

Cosine similarity → SimHash (random hyperplanes)

Euclidean distance → p-stable distributions

Hamming distance → bit sampling

§10 — Eight Worked Numerical Examples

Worked Example 1 — 2-shingles & Jaccard of two short strings

Worked Example 2 — Build a 4×3 boolean matrix, compute all pairwise Jaccards

Worked Example 3 — Permutation-based MinHash

Worked Example 4 — Hash-function MinHash (the row-by-row update)

Worked Example 5 — Jaccard estimate from signatures

Worked Example 6 — LSH banding

Worked Example 7 — S-curve calibration (b=20b = 20, r=5r = 5)

Worked Example 8 — Threshold-driven (b, r) selection

§11 — Practice Questions (15)

§12 — Full Worked Answers

§13 — Ending Key Notes (Revision Cards)

§14 — Formula & Algorithm Reference

Algorithmic pseudocode (memorize)

Complexities

Connections to other weeks

Week 04 – 05 · Finding Similar Items (Shingling → MinHash → LSH)

Why this topic is exam-critical. The midterm pattern is unambiguous: the examiner loves multi-step numerical traces. Shingling → MinHash → LSH is the *single longest pipeline* in the syllabus — and Dr. Imran has a documented habit of asking students to (a) build a shingle set, (b) fill a signature matrix row-by-row using two hash functions, then (c) band the signatures into LSH buckets. If you can mechanically execute that pipeline on a 4-row × 3-column toy matrix, you have ~25–30 marks locked in. Master §5–§7 and §10 (worked examples) above all else.

§1 — Beginning Key Notes (Study Compass)

These are the ten load-bearing ideas you must walk into the exam owning. Every numerical question on similar-items reduces to applying one of them.

1. **The problem.** Given N documents, find every pair (D_i, D_j) whose textual similarity exceeds threshold s. Naïve pairwise comparison is $O(N^2)$ — infeasible for $N = 10^6$ (that is $\approx 5 \times 10^{11}$ comparisons). The whole pipeline exists to dodge this.
2. **The 3-step pipeline.** *Shingling* converts each document to a set of k-grams. *MinHashing* compresses each set to a small signature (length ~100). *Locality-Sensitive Hashing* uses banding to identify candidate pairs in $\sim O(N)$.
3. **k-shingle.** A length-k contiguous substring (or word k-gram) of the document. $k = 5$ for emails, $k = 8$–10 for full webpages. Choose k large enough that random documents do **not** share many shingles.
4. **Set-membership matrix.** Rows = unique shingles in the universe, columns = documents. Entry is 1 iff the shingle appears in the document. Always sparse — never store densely.
5. **Jaccard similarity.** $J(A, B) = |A \cap B| / |A \cup B|$. The default similarity for sets. Equals 1 iff identical, 0 iff disjoint.
6. **MinHash function.** Pick a permutation π of the rows; $h_\pi(C)$ is the row index of the **first 1** in column C under that permutation. Repeat with ~ 100 permutations to build the signature column.
7. **The MinHash theorem.** $\Pr[h_\pi(A) = h_\pi(B)] = J(A, B)$. The probability two columns *minhash to the same value* equals their Jaccard. This is why MinHash is similarity-preserving compression.
8. **Implementing MinHash without permutations.** Use a hash family $h_i(x) = (a_i x + b_i) \bmod p$ instead of explicit permutations. For each row r: compute $h_i(r)$ for each i; for every column c with a 1 in row r, update $M[i, c] \leftarrow \min(M[i, c], h_i(r))$. Initialize $M[i, c] = \infty$.

9. **LSH banding.** Split the signature matrix (length n) into b bands of r rows each, $n = b \cdot r$. Hash each (band, column) chunk into a bucket. Two columns become a **candidate pair** iff they collide in ≥ 1 band.
10. **The S-curve.** $\Pr[\text{candidate pair} \mid \text{Jaccard } s] = 1 - (1 - s^r)^b$. Sharp transition near threshold $t \approx (1/b)^{1/r}$. Tune b, r to get the false-positive / false-negative balance you want.

> **The single biggest exam pattern.** Build a small set-membership matrix, then walk the row-by-row MinHash algorithm to fill the signature matrix, then band the signatures and report which document pairs are candidates. This is a guaranteed long-form question — usually 15–25 marks.

---

§2 — The Near-Duplicate Problem: Why Pairwise Comparison Doesn't Scale

Setting. You have N documents — webpages, emails, news articles, customer-purchase sets, image fingerprints — and you want to find every pair (D_i, D_j) such that some similarity score $\text{sim}(D_i, D_j) \geq s$. This appears constantly in real systems:

- **Plagiarism / mirror detection.** Are two webpages near-duplicates of each other?
- **News-article clustering.** Group AP/Reuters retellings of the same story.
- **Collaborative filtering.** Customers who purchased *similar product sets* should be similarly recommended.
- **Image deduplication.** Find images with similar feature vectors.
- **Genome / DNA assembly.** Reads that overlap (share many k-mers) belong to the same region.

Why naïve scanning fails. Compare every pair of documents:

$$\text{Number of pairs} = \binom{N}{2} = \frac{N(N-1)}{2} \approx \frac{N^2}{2}$$

For $N = 10^6$ that is 5×10^{11} comparisons. Even at 10 ns per comparison, you need **~50,000 seconds ≈ 14 hours** — and that's just to compare, not to read documents from disk. For $N = 10^9$ webpages it is hopeless.

The asymmetry the pipeline exploits. Most pairs of documents are *not* similar. We want the algorithm's running time to scale with the number of *truly similar* pairs, not the number of all pairs. Locality-Sensitive Hashing is exactly that algorithm: only candidate pairs (very likely to be similar) are explicitly compared.

> **EXAM TRAP — "linear time" myth.** LSH gives near-linear time *in expectation when truly similar pairs are sparse*. If you feed it a dataset where every pair is similar, the algorithm degenerates to pairwise comparison. The pipeline is designed for the typical case in which similar pairs are a tiny minority — exactly the web-search and document-deduplication setting.

Three obstacles the pipeline solves:

1. Documents are strings of variable length; we need a *fixed* representation. → **Shingling.**
2. The shingle universe is huge (~10710^7 unique 5-shingles in English); set sizes can be tens of thousands. Storing all sets is expensive. → **MinHash compression.**
3. Even with compressed signatures, comparing every pair of signatures is O(N2)O(N^2). → **LSH banding.**

---

§3 — Step 1: Shingling (Documents → Sets)

The first step converts each document into a *set* — discarding ordering, retaining the units of meaning that matter for similarity.

Definition — k-shingle (k-gram)

A **k-shingle** of a document DD is a sequence of kk contiguous tokens that appears in DD. Tokens can be:

- **Characters** (most common in slides and exam questions). Every string of kk consecutive characters is a shingle.
- **Words.** Common in NLP — "kk-grams" in language-model vocabulary.
- **Bytes.** For binary documents.

Formally, if D=c1c2...cLD = c_1 c_2 \ldots c_L is a sequence of LL tokens, the set of kk-shingles is

Sk(D)={cici+1...ci+k−1:1≤i≤L−k+1} S_k(D) = \{ c_i c_{i+1} \ldots c_{i+k-1} : 1 \leq i \leq L - k + 1 \}

Worked sub-example. D=abcabD = \texttt{abcab}, k=2k = 2.

Slide a window of length 2 across DD: ab\texttt{ab}, bc\texttt{bc}, ca\texttt{ca}, ab\texttt{ab}. Repeat shingles deduplicate to {ab,bc,ca}\{ \texttt{ab}, \texttt{bc}, \texttt{ca} \}. So |S2(D)|=3|S_2(D)| = 3.

Choosing k

The choice of kk controls how easily two unrelated documents accidentally share shingles.

Document length	Suggested	Why
Short (emails, tweets)	k=5k = 5	Short docs already have few unique shingles; small kk keeps them distinct enough.
Medium (articles)	k=7k = 7–88	Enough specificity to make accidental collisions rare.
Long (webpages)	k=9k = 9–1010	Larger universe; random documents almost never share a 10-shingle.

The rule of thumb from MMDS: pick kk so the **probability that a random kk-shingle appears in any given document is small.** For English text with ~26 letters but realistic frequency distributions, k=5k = 5

already gives a universe of $\sim 27^5 \approx 1.4 \times 10^7$ — orders of magnitude more than typical document sizes.

> **EXAM TRAP — too-small k.** If $k=1$, every English document shares all 26 letters → Jaccard similarity is artificially high for random pairs. If k is too large, even genuinely similar documents miss shared shingles because of small differences. The slides specifically warn: k must be "large enough or most documents will have most shingles."

Hashing Shingles to Compress

Long shingles (e.g. 10 character strings) eat memory. Solution: hash each shingle to a 4-byte (32-bit) integer.

$S_k(D) \xrightarrow{h} \{ h(s) : s \in S_k(D) \}$

Now each set is a set of *integers*. There is a tiny chance two distinct shingles collide to the same hash, which slightly inflates Jaccard — negligible at 32-bit hash widths.

Slide example. $D = \texttt{abcab}$, $k = 2$, $S_2(D) = \{ \texttt{ab}, \texttt{bc}, \texttt{ca} \}$. Hash: - $h(\texttt{ab}) = 1$ - $h(\texttt{bc}) = 5$ - $h(\texttt{ca}) = 7$

Compressed representation: $h(D_1) = \{ 1, 5, 7 \}$.

Sets → Boolean Matrix

Stack all the shingle sets into a **set-membership matrix** (also called the *characteristic matrix*):

- Rows = unique shingles in the universe (across all documents).
- Columns = documents.
- $M[r,c] = 1$ iff shingle r appears in document c, else 0.

Tiny example. Three documents, universe of 5 shingles $\{s_1, \ldots, s_5\}$:

s_1	1	0	1
s_2	1	1	0
s_3	0	1	1
s_4	0	0	1
s_5	1	1	0

Reading column D_1 tells you $D_1 = \{s_1, s_2, s_5\}$.

This matrix is **always sparse** — most shingles appear in only a handful of documents. Production systems never store it densely; they store each column as a sorted list of row indices. The dense table is only a teaching device.

> **EXAM TRAP — character vs. word shingles.** Read the question carefully. "2-shingles of `abcab`" usually means *character* 2-shingles. "2-shingles of the sentence 'the cat sat'" usually means *word* 2-shingles: {"the cat","cat sat"}\{\text{"the cat"}, \text{"cat sat"}\}. Always state which interpretation you used.

---

§4 — Jaccard Similarity (The Right Baseline for Sets)

Once each document is a set of (possibly hashed) shingles, we need a similarity measure between *sets*. The default is **Jaccard**:

J(A,B)=|A∩B||A∪B| J(A, B) = \frac{|A \cap B|}{|A \cup B|}

Properties

- **Range.** J(A,B)∈[0,1]J(A, B) \in [0, 1]. J=1J = 1 iff A=BA = B. J=0J = 0 iff A∩B=∅A \cap B = \emptyset.
- **Symmetric.** J(A,B)=J(B,A)J(A, B) = J(B, A).
- **Jaccard distance.** dJ(A,B)=1−J(A,B)d_J(A, B) = 1 - J(A, B) is a true metric (satisfies triangle inequality).
- **Insensitive to set sizes.** If A⊆BA \subseteq B but |A|≪|B||A| \ll |B|, J=|A|/|B|J = |A|/|B| — small. Jaccard rewards genuine *overlap*, not just one-way containment.

Slide Example — three documents

From the boolean matrix in §3:

- D1={s1,s2,s5}D_1 = \{s_1, s_2, s_5\}
- D2={s2,s3,s5}D_2 = \{s_2, s_3, s_5\}
- D3={s1,s3,s4}D_3 = \{s_1, s_3, s_4\}

Compute pairwise Jaccards:

- J(D1,D2)=|{s2,s5}|/|{s1,s2,s3,s5}|=2/4=0.50J(D_1, D_2) = |\{s_2, s_5\}| / |\{s_1, s_2, s_3, s_5\}| = 2/4 = 0.50
- J(D1,D3)=|{s1}|/|{s1,s2,s3,s4,s5}|=1/5=0.20J(D_1, D_3) = |\{s_1\}| / |\{s_1, s_2, s_3, s_4, s_5\}| = 1/5 = 0.20
- J(D2,D3)=|{s3}|/|{s1,s2,s3,s4,s5}|=1/5=0.20J(D_2, D_3) = |\{s_3\}| / |\{s_1, s_2, s_3, s_4, s_5\}| = 1/5 = 0.20

Why Jaccard?

Two reasons it is the canonical baseline for set similarity:

1. **It treats both documents symmetrically.** Cosine similarity on bag-of-words can be inflated by a single very long document. Jaccard cannot — adding shingles to AA that are not in BB pushes JJ down.
2. **It admits an efficient probabilistic estimator** — namely MinHash (next section). No other set similarity has such an elegant compression trick at the same level of mathematical simplicity.

> **EXAM TRAP — "size of intersection / size of one set."** Several students confuse Jaccard with overlap coefficient $|A \cap B| / \min(|A|, |B|)$ or with the Sørensen–Dice coefficient $2|A \cap B| / (|A| + |B|)$. **Jaccard is intersection over UNION.** Always.

Reading Jaccard Off the Boolean Matrix

For two columns C_1 and C_2 classify each row by its (C_1, C_2) pattern:

Type		
A	1	1
B	1	0
C	0	1
D	0	0

Let a, b, c, d be the counts. Then

$$J(C_1, C_2) = \frac{a}{a + b + c}$$

Type-D rows are ignored — they contribute nothing to either intersection or union. This bookkeeping becomes the basis of the MinHash theorem in §5.

---

§5 — Step 2: MinHashing (Sets → Short Signatures)

Storing the full shingle set per document is wasteful: a webpage might have 50,000 shingles. We want a tiny signature (~100 integers) that *preserves Jaccard similarity in expectation*.

The MinHash Idea

Pick a random permutation π of the rows of the boolean matrix. For each column C define

$$h_\pi(C) = \min \{ \pi(r) : C[r] = 1 \}$$

i.e. **the index of the first row (in the permuted order) that has a 1 in column** C. That single integer is the column's MinHash under permutation π.

Apply ~ 100 different permutations to each column. Stack the results vertically:

Hash function			
h_1	2	1	2
h_2	1	4	1
$\vdots$			
h_{100}	7	3	7

This is the **signature matrix** M_{sig}. Columns = documents (same as before); rows = hash-function indices (much smaller — typically 100). Each column is now a length-100 integer vector.

The MinHash Theorem (the killer lemma)

> $\Pr_\pi[h_\pi(C_1) = h_\pi(C_2)] = J(C_1, C_2)$

Proof sketch. Classify rows by type as in §4: a rows are type A (both 1), b type B (only C_1 is 1), c type C (only C_2 is 1). Type-D rows have no 1s and are irrelevant.

Under a random permutation, the **first row of type A or B or C** (i.e., the first row where at least one of C_1, C_2 has a 1) is equally likely to be any of the $a + b + c$ such rows. The two columns minhash to the *same* value precisely when that first non-type-D row is type A. So

$$\Pr[h_\pi(C_1) = h_\pi(C_2)] = \frac{a}{a + b + c} = J(C_1, C_2)$$

QED.

Why It Works in Practice (Estimation)

We don't compute one MinHash — we compute $n \approx 100$ independent ones. The estimator

$$\hat{J}(C_1, C_2) = \frac{\#\{i : M_{\text{sig}}[i, C_1] = M_{\text{sig}}[i, C_2]\}}{n}$$

(the fraction of signature rows on which the two columns agree) is an **unbiased estimator** of the true Jaccard. By the law of large numbers, as $n \to \infty$ the estimate converges to the true J. With $n = 100$ the standard error is roughly $1/\sqrt{n} = 0.1$ — small enough for ranking and candidate generation.

> **EXAM TRAP — MinHash uses MIN, not max.** A surprisingly common error. The signature is the *minimum* row index under the permutation. Calling it "MaxHash" or computing maxima will silently produce a number unrelated to Jaccard.

Slide Example (Worked) — 7 rows × 4 columns, 3 permutations

The slides give this matrix (rows are shingles, cols D_1–D_4):

Row				
1	1	0	1	0
2	1	0	0	1
3	0	1	0	1
4	0	1	0	1
5	0	1	0	1
6	1	0	1	0
7	1	0	1	0

Three permutations are given on the original rows (row → new position):

- $\pi 1$\pi_1: rows 1,2,3,4,5,6,7↦2,3,7,6,1,5,41, 2, 3, 4, 5, 6, 7 \mapsto 2, 3, 7, 6, 1, 5, 4
- $\pi 2$\pi_2: ↦4,2,1,3,6,7,5\mapsto 4, 2, 1, 3, 6, 7, 5
- $\pi 3$\pi_3: ↦3,4,7,2,6,1,5\mapsto 3, 4, 7, 2, 6, 1, 5

(The exact permutations match the slide example.) Reading the slide's resulting signature matrix:

Hash				
h1h_1	2	1	2	1
h2h_2	2	1	4	1
h3h_3	1	2	1	2

Now estimate Jaccards from signatures: - Ĵ(D1,D4)=0/3=0\hat{J}(D_1, D_4) = 0/3 = 0. **True**: D1={1,2,6,7}D_1 = \{1,2,6,7\}, D4={2,3,4,5}D_4 = \{2,3,4,5\}, J=1/7≈0.14J = 1/7 \approx 0.14. (3 hash functions is too few — this is the small-nn noise.) - Ĵ(D1,D3)=2/3≈0.67\hat{J}(D_1, D_3) = 2/3 \approx 0.67. **True**: D3={1,6,7}D_3 = \{1, 6, 7\}, J(D1,D3)=3/4=0.75J(D_1, D_3) = 3/4 = 0.75. Reasonably close. - Ĵ(D2,D4)=2/3≈0.67\hat{J}(D_2, D_4) = 2/3 \approx 0.67. **True**: D2={3,4,5}D_2 = \{3, 4, 5\}, J(D2,D4)=3/4=0.75J(D_2, D_4) = 3/4 = 0.75. Close.

The slide remark: "Sim from signatures = 2/2" reflects the slides displaying just the *first 2 rows* of MsigM_{\text{sig}} — the third row was added later. The takeaway: **estimates from a 3-row signature are noisy; in production we use 100+.**

§6 — Implementing MinHash Without Permutations

A real permutation of 10910^9 rows is unworkable: storing it requires 10910^9 entries, and accessing rows in permuted order causes catastrophic memory thrashing. The trick is to **simulate** nn permutations using nn hash functions.

The Hash Family

Pick n hash functions of the form

$h_i(x) = (a_i x + b_i) \bmod p \pmod{M}$

where a_i, b_i are random constants, p is a prime $\geq M$, and M is the number of rows. Each h_i approximates a random permutation: $h_i(x)$ tells you "the new position of row x under the i-th permutation."

The Algorithm (slide form, memorize this)

```
 Initialize M[i, c] = ∞ for all i ∈ {1..n}, c ∈ {1..N}
for each row r = 1, 2, ..., M:
    for each hash function i = 1, 2, ..., n:
        compute v_i = h_i(r)
    for each column c with C[r] = 1:
        for each hash function i:
            if v_i < M[i, c]:
                M[i, c] = v_i
```

Reading this carefully: we sweep through rows in original order (one disk pass!). For each row, we ask "which columns have a 1 in this row?" For each such column, we update its i-th signature slot to the **minimum** of (current value, h_i of this row). At the end, $M[i,c]$ is exactly $\min \{ h_i(r) : C[r] = 1 \}$ — the simulated i-th MinHash of column c.

Step-by-Step Trace (slide example: 5 rows, 2 columns, 2 hashes)

Setup. Two hash functions: - $h_1(x) = x \bmod 5$ - $h_2(x) = (2x + 1) \bmod 5$

Boolean matrix:

Row		
1	1	0
2	0	1
3	1	1
4	1	0
5	0	1

Initial signature matrix:

h_1	∞	∞
h_2	∞	∞

Row r = 1. h1(1)=1mod5=1h_1(1) = 1 \bmod 5 = 1. h2(1)=3mod5=3h_2(1) = 3 \bmod 5 = 3. - C1C_1 has a 1 in row 1: update M[1,C1]=min(∞,1)=1M[1, C_1] = \min(\infty, 1) = 1, M[2,C1]=min(∞,3)=3M[2, C_1] = \min(\infty, 3) = 3. - C2C_2 has a 0: skip.

h1h_1	1	∞\infty
h2h_2	3	∞\infty

Row r = 2. h1(2)=2h_1(2) = 2. h2(2)=5mod5=0h_2(2) = 5 \bmod 5 = 0. - C1C_1 has 0: skip. - C2C_2 has 1: update M[1,C2]=min(∞,2)=2M[1, C_2] = \min(\infty, 2) = 2, M[2,C2]=min(∞,0)=0M[2, C_2] = \min(\infty, 0) = 0.

h1h_1	1	2
h2h_2	3	0

Row r = 3. h1(3)=3h_1(3) = 3. h2(3)=7mod5=2h_2(3) = 7 \bmod 5 = 2. - C1C_1 has 1: M[1,C1]=min(1,3)=1M[1, C_1] = \min(1, 3) = 1 (no change). M[2,C1]=min(3,2)=2M[2, C_1] = \min(3, 2) = 2 (update). - C2C_2 has 1: M[1,C2]=min(2,3)=2M[1, C_2] = \min(2, 3) = 2 (no change). M[2,C2]=min(0,2)=0M[2, C_2] = \min(0, 2) = 0 (no change).

h1h_1	1	2
h2h_2	2	0

Row r = 4. h1(4)=4h_1(4) = 4. h2(4)=9mod5=4h_2(4) = 9 \bmod 5 = 4. - C1C_1 has 1: M[1,C1]=min(1,4)=1M[1, C_1] = \min(1, 4) = 1 (no change). M[2,C1]=min(2,4)=2M[2, C_1] = \min(2, 4) = 2 (no change). - C2C_2 has 0: skip.

h1h_1	1	2
h2h_2	2	0

Row r = 5. h1(5)=5mod5=0h_1(5) = 5 \bmod 5 = 0. h2(5)=11mod5=1h_2(5) = 11 \bmod 5 = 1. - C1C_1 has 0: skip. - C2C_2 has 1: M[1,C2]=min(2,0)=0M[1, C_2] = \min(2, 0) = 0 (update). M[2,C2]=min(0,1)=0M[2, C_2] = \min(0, 1) = 0 (no change).

Final signature matrix:

h1h_1	1	0
h2h_2	2	0

Hmm — wait. The slide ends at row 5 with M=(1020)M = \begin{pmatrix} 1 & 0 \\ 2 & 0 \end{pmatrix}. Verify by hand from the definition: h1h_1 values for rows where C1=1C_1 = 1 are h1(1)=1,h1(3)=3,h1(4)=4h_1(1) = 1, h_1(3) = 3, h_1(4) = 4. Min = 1. ✓. For C2=1C_2 = 1: rows 2, 3, 5; values h1(2)=2,h1(3)=3,h1(5)=0h_1(2) = 2, h_1(3) = 3, h_1(5) = 0. Min = 0. ✓.

(The original slide stops at the **second-to-last row** displaying M=(1020)M = \begin{pmatrix} 1 & 0 \\ 2 & 0 \end{pmatrix} which matches.)

> **EXAM TRAP — initialize to ∞.** Forgetting this step leads to garbage signatures. Always start every M[i,c]M[i, c] at "infinity" (in practice, the largest representable integer); only then does the running min update correctly.

> **EXAM TRAP — only update when the row has a 1.** The whole point of MinHash is "first row in which the column has a 1." If you update the signature for every row regardless of C[r]C[r], you compute the minimum over *all* rows — which is the same number for every column and tells you nothing.

Why hih_i ≈ random permutation

A truly random permutation is uniformly distributed over the M!M! orderings. The hash family hi(x)=(aix+bi)modph_i(x) = (a_i x + b_i) \bmod p is **pairwise-independent** (universal hashing): for any x≠yx \neq y, the joint distribution of (hi(x),hi(y))(h_i(x), h_i(y)) is approximately uniform. That is enough for the MinHash theorem to hold *in expectation*. It is not perfect — for very small MM collisions may distort things — but for MM in the millions it is indistinguishable from a real permutation.

---

§7 — Step 3: Locality-Sensitive Hashing (LSH)

Now every document is a length-nn integer signature (typically n=100n = 100). Comparing all (N2) \binom{N}{2} signature pairs is still O(N2)O(N^2) — we have shrunk each comparison from "compare two huge sets" to "compare two length-100 vectors," but the pair count is unchanged. We need a way to **find candidate similar pairs without enumerating all pairs**. That is LSH.

Banding Construction

Divide the signature matrix's nn rows into bb contiguous **bands** of rr rows each, with n=b·rn = b \cdot r. For each band:

1. Take that band's slice of every column — a length-rr integer vector per document.
2. Hash that length-rr vector to a bucket id (using any standard hash, e.g. a tuple hash).

3. Two columns whose band-j slices hash to the same bucket are said to **collide in band** j.

A pair of documents is a **candidate pair** iff their signatures collide in **at least one** band.

Visual

```
Signature matrix (n rows, N columns):
  Band 1 (r rows) ──┐
  Band 2 (r rows)   │  → for each band, hash every column's slice
  Band 3 (r rows)   │     into a bucket; remember collisions
  ...               │
  Band b (r rows) ──┘
```

Why It Works

If two columns are *very* similar (Jaccard s close to 1), then most rows of their signatures match → most bands have all r rows matching → very high probability of at least one band collision.

If two columns are *very* dissimilar (s close to 0), then most rows of their signatures disagree → it is rare for all r rows of any band to match → low probability of even one band collision.

Probability Calculation

Let s be the Jaccard of two columns. Then by the MinHash theorem each signature row matches with probability s. For one specific band of r rows:

$$\Pr[\text{all } r \text{ rows match in this band}] = s^r$$

$$\Pr[\text{at least one row in this band differs}] = 1 - s^r$$

For the columns to be **non-candidates**, *every* band must have at least one differing row:

$$\Pr[\text{no band identical}] = (1 - s^r)^b$$

Therefore

$$\Pr[\text{candidate pair} \mid s] = 1 - (1 - s^r)^b \quad \boxed{\Pr[\text{candidate pair}] \mid s] = 1 - (1 - s^r)^b}$$

This is the famous **S-curve** — sigmoid-shaped, with a sharp transition near similarity $s \approx (1/b)^{1/r}$.

Numerical Verification (slide example: $b=20$, $r=5$)

Similarity				
0.2	0.00032	0.99968	0.9936	**0.006**
0.3	0.00243	0.99757	0.9526	**0.047**
0.4	0.01024	0.98976	0.8143	**0.186**
0.5	0.03125	0.96875	0.5298	**0.470**
0.6	0.07776	0.92224	0.1977	**0.802**
0.7	0.16807	0.83193	0.0247	**0.975**
0.8	0.32768	0.67232	0.00035	**0.9996**

So at threshold $s=0.8$: - True positives: 99.96% of truly similar pairs become candidates. - False positives: only 0.6% of $s=0.2$ pairs and 4.7% of $s=0.3$ pairs are candidates. Most non-similar pairs are filtered out.

What Bands Detect

Two columns become candidates iff they agree in **all r rows of some band**. The "agree" check is exact equality of two integer tuples — a single hash lookup per band. Total work per band:

$$N \text{ documents} \times O(r) \text{ hash work} = O(Nr)$$

Total work across b bands: $O(Nrb)=O(Nn)$. Then for each colliding-bucket pair, do an actual signature-similarity check — but only on candidates, which is a tiny fraction of all pairs.

> **EXAM TRAP — bands AND rows, don't confuse.** b = number of bands. r = rows per band. Total signature length $n=b \cdot r$. Many students write "b rows per band" or "r bands." Use the mnemonic *b is for bands, r is for rows*.

> **EXAM TRAP — "candidates collide in ≥ 1 band" not all bands.** A pair becomes a candidate as soon as ONE band matches. Requiring all bands to match would defeat the purpose — you would only catch identical signatures.

Why "Locality Sensitive"

A hash family $\mathcal{H}$ is called (d_1, d_2, p_1, p_2)-**locality-sensitive** if: for any two points x, y, $\Pr_{h \in \mathcal{H}}[h(x)=h(y)] \geq p_1$ when $\text{sim}(x,y) \geq d_1$, and $\leq p_2$ when $\text{sim}(x,y) \leq d_2$, with $p_1 > p_2$. MinHash + banding is exactly such a family for Jaccard similarity. The phrase "locality sensitive" refers to the fact that nearby (similar) points hash to the same bucket more often than far-apart ones.

§8 — Tuning b and r: The S-Curve Threshold

We don't usually pick b and r randomly. We pick them to **target a specific similarity threshold** and tolerate a specific false-positive / false-negative rate.

The Threshold Formula

The S-curve's inflection (50%-probability point) is approximately

$$t \approx \left(\frac{1}{b} \right)^{1/r}$$

Pairs with $s \gg t$ become candidates with probability $\to 1$; pairs with $s \ll t$ are filtered out with probability $\to 1$. Pairs near $s \approx t$ are uncertain — that's the transition region.

Worked: $b = 20$, $r = 5$, $n = 100$

$t \approx (1/20)^{1/5} = 0.05^{0.2}$. Compute $\log_{10}(0.05) = -1.301$, so $0.05^{0.2} = 10^{-0.260} \approx 0.55$. The threshold is around 0.55.

Looking at the S-curve table in §7, that matches: $\Pr[\text{candidate}]$ crosses 50% somewhere between $s = 0.5$ ($\Pr = 0.47$) and $s = 0.6$ ($\Pr = 0.80$). The half-probability point is near $s = 0.51$ — close to the formula's prediction.

Trade-off

Move	Effect on threshold	Effect on FPs	Effect on FNs
Increase b (more bands, fewer rows each)	t decreases	More FPs (more buckets to collide)	Fewer FNs
Increase r (fewer bands, more rows each)	t increases	Fewer FPs (harder to collide)	More FNs
Larger $n = b \cdot r$	sharper S-curve	both go down at fixed t	both go down

Engineering recipe. Decide your similarity threshold s^*. Pick b, r so that $(1/b)^{1/r} \approx s^*$. Empirically, $n \in [50, 200]$ rows is enough for a sharp curve.

> **EXAM TRAP — the threshold formula is approximate.** $t \approx (1/b)^{1/r}$, not $1/(b r)$ and not $1/(b+r)$. The exponent matters! At $b = 20, r = 5$, $1/(b r) = 0.01$ which is wildly wrong; the correct threshold is ~0.55.

> **EXAM TRAP — variance of Jaccard estimate from signatures.** With signature length n, the variance of $\hat{J}$ is approximately $J(1-J)/n$, so the standard error is $\sim 1/\sqrt{n}$. For $n=100$: SE ≈ 0.10. To halve the SE, you need 4× the signature length. This is why production setups use $n=200$ or more.

---

§9 — LSH for Other Distances (briefly)

MinHash is the LSH family for **Jaccard** similarity. The same banding idea adapts to other metrics if you can find an appropriate hash family.

Cosine similarity → SimHash (random hyperplanes)

For real-valued vectors in $\mathbb{R}^d$ with cosine similarity, the LSH family is **random hyperplane projection**:

1. Sample a random unit vector $\mathbf{h} \in \mathbb{R}^d$.
2. Define $h_{\mathbf{h}}(\mathbf{x}) = \text{sign}(\mathbf{x}^\top \mathbf{h})$ (a single bit: 0 or 1 depending on which side of the hyperplane $\mathbf{x}$ lies).

Theorem. $\Pr[h_{\mathbf{h}}(\mathbf{x}) = h_{\mathbf{h}}(\mathbf{y})] = 1 - \theta(\mathbf{x}, \mathbf{y}) / \pi$, where θ is the angle between them. Closely related to cosine similarity since $\cos\theta \approx 1 - 2\theta/\pi$ for small angles.

Use w random hyperplanes per "table" and build T such tables — exactly the banding analogue. The slide example with hyperplanes h_1, h_2, h_3 giving codes 100, 011, etc., is SimHash with $w=3$.

Euclidean distance → p-stable distributions

Project onto a random Gaussian direction and bucket by interval; smaller distances → more likely to fall in the same bucket. Used for nearest-neighbor search in $\mathbb{R}^d$.

Hamming distance → bit sampling

Sample random bit positions; two binary vectors hash to the same bucket iff they agree on the sampled bit. Very lightweight — used for bit-vector image hashes.

For the exam, the key facts: - **LSH is a family of techniques** parameterised by the similarity (or distance) metric. - **MinHash is the LSH family for Jaccard.** SimHash is the LSH family for cosine. Different distances → different hash families. - All of them slot into the same banding architecture from §7.

---

§10 — Eight Worked Numerical Examples

These are the kind of multi-step traces the examiner expects. Read each solution once, then close the document and reproduce on paper. Time yourself: 6–8 minutes per worked example.

Worked Example 1 — 2-shingles & Jaccard of two short strings

Problem. Compute the 2-shingles of D1=abcabD_1 = \texttt{abcab} and D2=abcbaD_2 = \texttt{abcba}. Compute J(D1,D2)J(D_1, D_2).

Step 1 — shingles of D1D_1. Slide window of length 2: ab\texttt{ab}, bc\texttt{bc}, ca\texttt{ca}, ab\texttt{ab}. Deduplicate: S(D1)={ab,bc,ca}S(D_1) = \{ \texttt{ab}, \texttt{bc}, \texttt{ca} \}.

Step 2 — shingles of D2D_2. ab\texttt{ab}, bc\texttt{bc}, cb\texttt{cb}, ba\texttt{ba}. Deduplicate: S(D2)={ab,bc,cb,ba}S(D_2) = \{ \texttt{ab}, \texttt{bc}, \texttt{cb}, \texttt{ba} \}.

Step 3 — intersection. S(D1)∩S(D2)={ab,bc}S(D_1) \cap S(D_2) = \{ \texttt{ab}, \texttt{bc} \}. |S∩S|=2|S \cap S| = 2.

Step 4 — union. S(D1)∪S(D2)={ab,bc,ca,cb,ba}S(D_1) \cup S(D_2) = \{ \texttt{ab}, \texttt{bc}, \texttt{ca}, \texttt{cb}, \texttt{ba} \}. |S∪S|=5|S \cup S| = 5.

Step 5 — Jaccard.

J(D1,D2)=25=0.40 J(D_1, D_2) = \frac{2}{5} = 0.40

> Sanity check: The strings share 2 characters in common positions; share both prefixes and the bigram “bc”; differ in the second half. Jaccard 0.4 feels right.

Worked Example 2 — Build a 4×3 boolean matrix, compute all pairwise Jaccards

Problem. Universe of 4 shingles {e1,e2,e3,e4}\{e_1, e_2, e_3, e_4\}. Three documents: - D1={e1,e3}D_1 = \{e_1, e_3\} - D2={e1,e2,e4}D_2 = \{e_1, e_2, e_4\} - D3={e2,e3,e4}D_3 = \{e_2, e_3, e_4\}

Build the boolean matrix and compute J(D1,D2)J(D_1, D_2), J(D1,D3)J(D_1, D_3), J(D2,D3)J(D_2, D_3).

Step 1 — boolean matrix.

e1e_1	1	1	0
e2e_2	0	1	1
e3e_3	1	0	1
e4e_4	0	1	1

Step 2 — Jaccard of (D1, D2). D1∩D2={e1}D_1 \cap D_2 = \{e_1\} (size 1). D1∪D2={e1,e2,e3,e4}D_1 \cup D_2 = \{e_1, e_2, e_3, e_4\} (size 4). J=1/4=0.25J = 1/4 = 0.25.

Step 3 — Jaccard of (D1, D3). D1∩D3={e3}D_1 \cap D_3 = \{e_3\} (size 1). D1∪D3={e1,e2,e3,e4}D_1 \cup D_3 = \{e_1, e_2, e_3, e_4\} (size 4). J=1/4=0.25J = 1/4 = 0.25.

Step 4 — Jaccard of (D2, D3). D2∩D3={e2,e4}D_2 \cap D_3 = \{e_2, e_4\} (size 2). D2∪D3={e1,e2,e3,e4}D_2 \cup D_3 = \{e_1, e_2, e_3, e_4\} (size 4). J=2/4=0.50J = 2/4 = 0.50.

Summary table.

Pair			Jaccard
(D1,D2)(D_1, D_2)	1	4	0.25
(D1,D3)(D_1, D_3)	1	4	0.25
(D2,D3)(D_2, D_3)	2	4	0.50

So D2D_2 and D3D_3 are the closest pair.

Worked Example 3 — Permutation-based MinHash

Problem. Use the matrix from WE 2 with three explicit permutations of the rows. Fill the 3×3 signature matrix step by step, then verify the MinHash–Jaccard relationship.

Permutations (mapping original row → its position in the permutation): - π1\pi_1: e1→3e_1 \to 3, e2→1e_2 \to 1, e3→4e_3 \to 4, e4→2e_4 \to 2. (i.e. permuted order is e2,e4,e1,e3e_2, e_4, e_1, e_3.) - π2\pi_2: e1→2e_1 \to 2, e2→4e_2 \to 4, e3→1e_3 \to 1, e4→3e_4 \to 3. (permuted order: e3,e1,e4,e2e_3, e_1, e_4, e_2.) - π3\pi_3: e1→4e_1 \to 4, e2→2e_2 \to 2, e3→3e_3 \to 3, e4→1e_4 \to 1. (permuted order: e4,e2,e3,e1e_4, e_2, e_3, e_1.)

Step 1 — column D1={e1,e3}D_1 = \{e_1, e_3\}**.** - Under π1\pi_1, positions of D1D_1's 1s are π1(e1)=3\pi_1(e_1) = 3, π1(e3)=4\pi_1(e_3) = 4. Min = **3**. - Under π2\pi_2, positions are π2(e1)=2\pi_2(e_1) = 2, π2(e3)=1\pi_2(e_3) = 1. Min = **1**. - Under π3\pi_3, positions are π3(e1)=4\pi_3(e_1) = 4, π3(e3)=3\pi_3(e_3) = 3. Min = **3**.

Step 2 — column D2={e1,e2,e4}D_2 = \{e_1, e_2, e_4\}**.** - π1\pi_1: positions 3,1,23, 1, 2. Min = **1**. - π2\pi_2: positions 2,4,32, 4, 3. Min = **2**. - π3\pi_3: positions 4,2,14, 2, 1. Min = **1**.

Step 3 — column D3={e2,e3,e4}D_3 = \{e_2, e_3, e_4\}**.** - π1\pi_1: positions 1,4,21, 4, 2. Min = **1**. - π2\pi_2: positions 4,1,34, 1, 3. Min = **1**. - π3\pi_3: positions 2,3,12, 3, 1. Min = **1**.

Signature matrix.

h1h_1	3	1	1
h2h_2	1	2	1
h3h_3	3	1	1

Step 4 — estimate Jaccards from signatures. - J^(D1,D2)\hat{J}(D_1, D_2): rows where they agree — none. J^=0/3=0.00\hat{J} = 0/3 = 0.00. True = 0.25 (small-nn noise). - J^(D1,D3)\hat{J}(D_1, D_3): rows where agree — none. J^=0/3=0.00\hat{J} = 0/3 = 0.00. True = 0.25. - J^(D2,D3)\hat{J}(D_2, D_3): agree in h1h_1 (1=1) and h3h_3 (1=1). J^=2/3≈0.67\hat{J} = 2/3 \approx 0.67. True = 0.50.

The estimates are off because we used only 3 permutations. With 100 the average error would be ~0.05.

Worked Example 4 — Hash-function MinHash (the row-by-row update)

Problem. Same matrix as WE 2 but using 5 rows instead of 4 (call them $r1,\ldots,r5r_1, \ldots, r_5$ with the boolean matrix below). Hash functions $h1(x)=(x+1)mod5h_1(x) = (x + 1) \bmod 5$, $h2(x)=(3x+1)mod5h_2(x) = (3x + 1) \bmod 5$. Treat row rir_i as integer ii in the hash inputs. Run the algorithm row-by-row.

Boolean matrix:

Row			
1	1	0	0
2	0	1	1
3	1	1	0
4	0	0	1
5	1	0	1

Initial signature matrix.

$h1h_1$	∞	∞	∞
$h2h_2$	∞	∞	∞

Row r = 1. $h1(1)=2mod5=2h_1(1) = 2 \bmod 5 = 2$. $h2(1)=4mod5=4h_2(1) = 4 \bmod 5 = 4$. - $D1D_1$ has 1: $M[1,D1]\leftarrow min(\infty,2)=2M[1, D_1] \leftarrow \min(\infty, 2) = 2$. $M[2,D1]\leftarrow min(\infty,4)=4M[2, D_1] \leftarrow \min(\infty, 4) = 4$. - $D2,D3D_2, D_3$ have 0: skip.

$h1h_1$	2	∞	∞
$h2h_2$	4	∞	∞

Row r = 2. $h1(2)=3mod5=3h_1(2) = 3 \bmod 5 = 3$. $h2(2)=7mod5=2h_2(2) = 7 \bmod 5 = 2$. - $D1D_1$ has 0: skip. - $D2D_2$ has 1: $M[1,D2]\leftarrow min(\infty,3)=3M[1, D_2] \leftarrow \min(\infty, 3) = 3$. $M[2,D2]\leftarrow min(\infty,2)=2M[2, D_2] \leftarrow \min(\infty, 2) = 2$. - $D3D_3$ has 1: $M[1,D3]\leftarrow min(\infty,3)=3M[1, D_3] \leftarrow \min(\infty, 3) = 3$. $M[2,D3]\leftarrow min(\infty,2)=2M[2, D_3] \leftarrow \min(\infty, 2) = 2$.

$h1h_1$	2	3	3
$h2h_2$	4	2	2

Row r = 3. h1(3)=4mod5=4h_1(3) = 4 \bmod 5 = 4. h2(3)=10mod5=0h_2(3) = 10 \bmod 5 = 0. - D1D_1 has 1: M[1,D1]=min(2,4)=2M[1, D_1] = \min(2, 4) = 2 (no change). M[2,D1]=min(4,0)=0M[2, D_1] = \min(4, 0) = 0 (update). - D2D_2 has 1: M[1,D2]=min(3,4)=3M[1, D_2] = \min(3, 4) = 3 (no change). M[2,D2]=min(2,0)=0M[2, D_2] = \min(2, 0) = 0 (update). - D3D_3 has 0: skip.

h1h_1	2	3	3
h2h_2	0	0	2

Row r = 4. h1(4)=5mod5=0h_1(4) = 5 \bmod 5 = 0. h2(4)=13mod5=3h_2(4) = 13 \bmod 5 = 3. - D1,D2D_1, D_2 have 0: skip. - D3D_3 has 1: M[1,D3]=min(3,0)=0M[1, D_3] = \min(3, 0) = 0 (update). M[2,D3]=min(2,3)=2M[2, D_3] = \min(2, 3) = 2 (no change).

h1h_1	2	3	0
h2h_2	0	0	2

Row r = 5. h1(5)=6mod5=1h_1(5) = 6 \bmod 5 = 1. h2(5)=16mod5=1h_2(5) = 16 \bmod 5 = 1. - D1D_1 has 1: M[1,D1]=min(2,1)=1M[1, D_1] = \min(2, 1) = 1 (update). M[2,D1]=min(0,1)=0M[2, D_1] = \min(0, 1) = 0 (no change). - D2D_2 has 0: skip. - D3D_3 has 1: M[1,D3]=min(0,1)=0M[1, D_3] = \min(0, 1) = 0 (no change). M[2,D3]=min(2,1)=1M[2, D_3] = \min(2, 1) = 1 (update).

Final signature matrix.

h1h_1	1	3	0
h2h_2	0	0	1

Sanity check. For D1={1,3,5}D_1 = \{1, 3, 5\}: h1h_1 values are {2,4,1}\{2, 4, 1\}, min = 1 ✓. h2h_2 values are {4,0,1}\{4, 0, 1\}, min = 0 ✓. For D3={2,4,5}D_3 = \{2, 4, 5\}: h1h_1 values are {3,0,1}\{3, 0, 1\}, min = 0 ✓. h2h_2 values are {2,3,1}\{2, 3, 1\}, min = 1 ✓.

Estimate Jaccards. - J^(D1,D2)=0/2=0\hat{J}(D_1, D_2) = 0/2 = 0 (rows: 1≠31 \neq 3, 0=00 = 0 — actually they agree on h2h_2). Re-check: h2(D1)=0,h2(D2)=0h_2(D_1) = 0, h_2(D_2) = 0. They DO agree. J^=1/2=0.50\hat{J} = 1/2 = 0.50. True J=|{3}|/|{1,2,3,5}|=1/4=0.25J = |\{3\}|/|\{1,2,3,5\}| = 1/4 = 0.25. Noise. - J^(D1,D3)=0/2=0\hat{J}(D_1, D_3) = 0/2 = 0. True J=|{5}|/|{1,2,3,4,5}|=1/5=0.20J = |\{5\}|/|\{1,2,3,4,5\}| = 1/5 = 0.20. Consistent with low. - J^(D2,D3)=0/2=0\hat{J}(D_2, D_3) = 0/2 = 0. True J=|{2}|/|{2,3,4}|=1/3≈0.33J = |\{2\}|/|\{2,3,4\}| = 1/3 \approx 0.33. Noise.

Two hash functions is too few to estimate reliably; the variance is huge.

Worked Example 5 — Jaccard estimate from signatures

Problem. Two documents have signature columns of length 100. They agree in 35 of those 100 positions. Estimate their Jaccard. What is the standard error?

Step 1. $\hat{J} = 35/100 = 0.35$.

Step 2 — standard error. Variance of $\hat{J}$ for $n = 100$ Bernoulli trials with success probability J:

$$\text{Var}(\hat{J}) = \frac{J(1-J)}{n} \approx \frac{0.35 \cdot 0.65}{100} = 0.002275$$

$$\text{SE} = \sqrt{0.002275} \approx 0.0477$$

Step 3 — interpretation. A 95% confidence interval is $\hat{J} \pm 1.96 \cdot \text{SE} \approx 0.35 \pm 0.094 = [0.256, 0.444]$. So the true Jaccard is plausibly anywhere from ~0.26 to ~0.44 — useful for ranking, less precise than a full Jaccard computation.

> To halve the standard error, you would need $n = 400$. Doubling signature length only reduces SE by factor $\sqrt{2}$.

Worked Example 6 — LSH banding

Problem. Signature matrix has 12 rows, $b = 3$ bands of $r = 4$ rows each. Five documents $D_1, \ldots, D_5$ with signatures:

Row					
1	5	5	1	5	7
2	8	8	2	8	4
3	3	3	9	3	6
4	7	7	4	7	1
5	2	6	2	2	3
6	9	1	9	9	8
7	4	5	4	4	0
8	1	0	1	1	5
9	6	2	9	4	9
10	0	7	1	8	2
11	5	3	5	0	7
12	8	9	5	6	4

Identify candidate pairs.

Step 1 — Band 1 (rows 1–4). - D1D_1 slice: (5, 8, 3, 7). - D2D_2 slice: (5, 8, 3, 7). **Same as** D1D_1**.** - D3D_3 slice: (1, 2, 9, 4). - D4D_4 slice: (5, 8, 3, 7). **Same as** D1,D2D_1, D_2**.** - D5D_5 slice: (7, 4, 6, 1).

Band 1 collisions: {D1,D2,D4}\{D_1, D_2, D_4\} all share the bucket (5, 8, 3, 7). Pairs from band 1: (D1,D2)(D_1, D_2), (D1,D4)(D_1, D_4), (D2,D4)(D_2, D_4).

Step 2 — Band 2 (rows 5–8). - D1D_1: (2, 9, 4, 1). - D2D_2: (6, 1, 5, 0). - D3D_3: (2, 9, 4, 1). **Same as** D1D_1**.** - D4D_4: (2, 9, 4, 1). **Same as** D1,D3D_1, D_3**.** - D5D_5: (3, 8, 0, 5).

Band 2 collisions: {D1,D3,D4}\{D_1, D_3, D_4\} share bucket (2, 9, 4, 1). Pairs: (D1,D3)(D_1, D_3), (D1,D4)(D_1, D_4), (D3,D4)(D_3, D_4).

Step 3 — Band 3 (rows 9–12). - D1D_1: (6, 0, 5, 8). - D2D_2: (2, 7, 3, 9). - D3D_3: (9, 1, 5, 5). - D4D_4: (4, 8, 0, 6). - D5D_5: (9, 2, 7, 4).

Band 3: all five buckets are distinct → no collisions.

Step 4 — collect all candidate pairs (union over bands).

Band	Pairs
1	(D1,D2),(D1,D4),(D2,D4)(D_1, D_2), (D_1, D_4), (D_2, D_4)
2	(D1,D3),(D1,D4),(D3,D4)(D_1, D_3), (D_1, D_4), (D_3, D_4)
3	none

Candidate pair set: {(D1,D2),(D1,D3),(D1,D4),(D2,D4),(D3,D4)}\{(D_1, D_2), (D_1, D_3), (D_1, D_4), (D_2, D_4), (D_3, D_4)\}. Five pairs out of (52)=10\binom{5}{2} = 10 possible pairs — half are filtered out.

Notice D5D_5 is in **no** candidate pair: its signature differs from everyone else in every band. The LSH stage correctly excludes it from further consideration.

> **Reading the answer:** D4D_4 is in many candidate pairs because its signature is *very* close to D1,D2,D3D_1, D_2, D_3. (D2,D3)(D_2, D_3) never share a band, so they're excluded — a possible false negative if their true Jaccard is high.

Worked Example 7 — S-curve calibration (b=20b = 20, r=5r = 5)

Problem. For b=20,r=5b = 20, r = 5, compute Pr[candidate pair]\Pr[\text{candidate pair}] for s∈{0.2,0.4,0.6,0.8}s \in \{0.2, 0.4, 0.6, 0.8\} using 1−(1−s5)201 - (1 - s^5)^{20}. Comment on the shape of the S-curve.

Calculations:

0.2	0.00032	0.99968	≈0.9936\approx 0.9936	0.0064
0.4	0.01024	0.98976	≈0.8143\approx 0.8143	0.1857
0.6	0.07776	0.92224	≈0.1977\approx 0.1977	0.8023
0.8	0.32768	0.67232	≈0.000349\approx 0.000349	0.9997

Interpretation. Below similarity 0.4, the probability of becoming a candidate is < 19%. Above 0.6, it is > 80%. The transition is sharp — exactly the S-curve. The threshold is t≈(1/20)1/5=0.549t \approx (1/20)^{1/5} = 0.549, between 0.4 and 0.6 as expected.

Plot ASCII (similarity on x, probability on y):

```
  s    Pr   |
0.2 0.006  | ▌
0.3 0.047  | ▌▌
0.4 0.186  | ▌▌▌▌▌
0.5 0.470  | ▌▌▌▌▌▌▌▌▌▌▌▌
0.6 0.802  | ▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌
0.7 0.975  | ▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌
0.8 0.9997 | ▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌▌
```

The curve is flat for s≤0.3s \leq 0.3, steep around s=0.5s = 0.5–0.60.6, and saturated above 0.70.7.

Worked Example 8 — Threshold-driven (b, r) selection

Problem. Choose b,rb, r so that: - Pairs with J≥0.8J \geq 0.8 become candidates with prob ≥0.99\geq 0.99. - Pairs with J≤0.4J \leq 0.4 become candidates with prob ≤0.05\leq 0.05.

What signature length n=brn = b r is needed?

Step 1 — write the constraints.

1−(1−0.8r)b≥0.99⇒(1−0.8r)b≤0.01 1 - (1 - 0.8^r)^b \geq 0.99 \quad\Rightarrow\quad (1 - 0.8^r)^b \leq 0.01

1−(1−0.4r)b≤0.05⇒(1−0.4r)b≥0.95 1 - (1 - 0.4^r)^b \leq 0.05 \quad\Rightarrow\quad (1 - 0.4^r)^b \geq 0.95

Step 2 — try r=5r = 5**.**

Top constraint: (1−0.85)b=0.67232b≤0.01(1 - 0.8^5)^b = 0.67232^b \leq 0.01. Take logs: blog0.67232≤log0.01b \log 0.67232 \leq \log 0.01 → b⋅(−0.397)≤−4.605b \cdot (-0.397) \leq -4.605 → b≥11.6b \geq 11.6. So b≥12b \geq 12.

Bottom: (1−0.45)b=0.98976b≥0.95(1 - 0.4^5)^b = 0.98976^b \geq 0.95. b⋅log0.98976≥log0.95b \cdot \log 0.98976 \geq \log 0.95 → b⋅(−0.01030)≥−0.05129b \cdot (-0.01030) \geq -0.05129 → b≤4.98b \leq 4.98. So b≤4b \leq 4.

Conflict! With r=5r = 5, no value of bb works.

Step 3 — try r=4r = 4**.**

Top: $(1-0.84)^b=0.5904^b \leq 0.01$ (1 - 0.8^4)^b = 0.5904^b \leq 0.01. $b \geq \log 0.01/\log 0.5904=-4.605/-0.5269=8.74$ b \geq \log 0.01 / \log 0.5904 = -4.605 / -0.5269 = 8.74. So $b \geq 9$ b \geq 9.

Bottom: $(1-0.44)^b=0.9744^b \geq 0.95$ (1 - 0.4^4)^b = 0.9744^b \geq 0.95. $b \leq \log 0.95/\log 0.9744=-0.0513/-0.02594=1.98$ b \leq \log 0.95 / \log 0.9744 = -0.0513 / -0.02594 = 1.98. So $b \leq 1$ b \leq 1.

Still conflict.

Step 4 — try $r=6$ r = 6**.**

Top: $0.86=0.2621$ 0.8^6 = 0.2621. $(1-0.2621)^b=0.7379^b \leq 0.01$ (1 - 0.2621)^b = 0.7379^b \leq 0.01. $b \geq -4.605/-0.3041=15.14$ b \geq -4.605 / -0.3041 = 15.14. So $b \geq 16$ b \geq 16.

Bottom: $0.46=0.004096$ 0.4^6 = 0.004096. $(1-0.004096)^b=0.9959^b \geq 0.95$ (1 - 0.004096)^b = 0.9959^b \geq 0.95. $b \leq -0.0513/-0.004105=12.5$ b \leq -0.0513 / -0.004105 = 12.5. So $b \leq 12$ b \leq 12.

Still conflict.

Step 5 — try $r=8$ r = 8**.**

Top: $0.88=0.1678$ 0.8^8 = 0.1678. $(1-0.1678)^b=0.8322^b \leq 0.01$ (1 - 0.1678)^b = 0.8322^b \leq 0.01. $b \geq -4.605/-0.1837=25.07$ b \geq -4.605 / -0.1837 = 25.07. So $b \geq 26$ b \geq 26.

Bottom: $0.48=0.000655$ 0.4^8 = 0.000655. $(1-0.000655)^b=0.99934^b \geq 0.95$ (1 - 0.000655)^b = 0.99934^b \geq 0.95. $b \leq -0.0513/-0.000655=78.3$ b \leq -0.0513 / -0.000655 = 78.3. So $b \leq 78$ b \leq 78.

Both satisfied for $b \in [26,78]$ b \in [26, 78]**,** $r=8$ r = 8**.** Pick $b=26, r=8$ b = 26, r = 8 → signature length $n=208$ n = 208.

Step 6 — verification. - $1-(1-0.88)^{26}=1-0.832226=1-0.00827 \approx 0.992$ 1 - (1 - 0.8^8)^{26} = 1 - 0.8322^{26} = 1 - 0.00827 \approx 0.992 ✓ (≥ 0.99) - $1-(1-0.48)^{26}=1-0.9993426=1-0.9831 \approx 0.0169$ 1 - (1 - 0.4^8)^{26} = 1 - 0.99934^{26} = 1 - 0.9831 \approx 0.0169 ✓ (≤ 0.05)

So $(b=26, r=8)$ (b = 26, r = 8) with signature length 208 208 meets both targets.

Could we do it cheaper? $r=7$ r = 7 also works. Top: $0.87=0.2097$ 0.8^7 = 0.2097, $(0.7903)^b \leq 0.01$ (0.7903)^b \leq 0.01 → $b \geq 19.6$ b \geq 19.6, so $b \geq 20$ b \geq 20. Bottom: $0.47=0.001638$ 0.4^7 = 0.001638, $(0.99836)^b \geq 0.95$ (0.99836)^b \geq 0.95 → $b \leq 31.3$ b \leq 31.3. So $b \in [20,31]$ b \in [20, 31], $r=7$ r = 7 works with signature length $n \in [140,217]$ n \in [140, 217].

Pick the smallest: $b=20, r=7$ b = 20, r = 7, $n=140$ n = 140. Verify: $1-(1-0.87)^{20}=1-0.790320=1-0.0094=0.991$ 1 - (1 - 0.8^7)^{20} = 1 - 0.7903^{20} = 1 - 0.0094 = 0.991 ✓. $1-(1-0.47)^{20}=1-0.9983620=1-0.9677=0.0323$ 1 - (1 - 0.4^7)^{20} = 1 - 0.99836^{20} = 1 - 0.9677 = 0.0323 ✓. **Final answer:** $b=20, r=7, n=140$ b = 20, r = 7, n = 140**.**

---

§11 — Practice Questions (15)

Mix: 6 numerical traces, 5 conceptual short-answer, 4 MCQ / true-false. Time yourself: ~\sim 6 min per numerical, ~\sim 3 min per conceptual, ~\sim 1 min per MCQ.

Q1 [Numerical · 5 marks]. Compute the 3-shingles of $D=abcdabc$ D = \texttt{abcdabc}. List them as a set. How many distinct 3-shingles are there?

Q2 [Numerical · 6 marks]. Two documents have shingle sets A={p,q,r,s,t}A = \{p, q, r, s, t\}, B={q,s,t,u,v}B = \{q, s, t, u, v\}. (a) Compute |A∩B||A \cap B|, |A∪B||A \cup B|, J(A,B)J(A, B). (b) Compute the Jaccard distance dJ(A,B)d_J(A, B).

Q3 [Numerical · 8 marks]. Build the boolean matrix for documents D1={a,c,e}D_1 = \{a, c, e\}, D2={b,c,d}D_2 = \{b, c, d\}, D3={a,b,e}D_3 = \{a, b, e\}, D4={c,d,e}D_4 = \{c, d, e\} over the universe {a,b,c,d,e}\{a, b, c, d, e\}. Compute all (42)=6\binom{4}{2} = 6 pairwise Jaccard similarities. Which pair is most similar?

Q4 [Numerical · 10 marks]. Use the boolean matrix in Q3 plus two hash functions h1(x)=(x+2)mod5h_1(x) = (x + 2) \bmod 5 and h2(x)=(3x+4)mod5h_2(x) = (3x + 4) \bmod 5 (treat row a=1,b=2,c=3,d=4,e=5a = 1, b = 2, c = 3, d = 4, e = 5). Run the row-by-row MinHash algorithm and produce the 2×42 \times 4 signature matrix. Show every update.

Q5 [Numerical · 6 marks]. Two columns have a 100-row signature. They agree in 78 rows. (a) Estimate JJ. (b) Compute the standard error. (c) Construct a 95% confidence interval for JJ.

Q6 [Numerical · 8 marks]. A signature matrix has 8 rows and the following 4 columns. Use b=4b = 4 bands of r=2r = 2 rows each. Identify candidate pairs.

Row				
1	1	2	1	3
2	4	5	4	6
3	7	7	8	9
4	1	1	2	3
5	9	9	9	4
6	0	0	1	5
7	3	6	3	6
8	2	4	2	4

Q7 [Concept · 4 marks]. State the MinHash theorem and explain in your own words why it holds. Use the type-A/B/C/D row classification.

Q8 [Concept · 4 marks]. Suppose you set k=2k = 2 for shingling English-language webpages. What goes wrong? Give a specific example of two unrelated documents that would appear artificially similar.

Q9 [Concept · 4 marks]. Compare and contrast the role of MinHash and LSH banding. Could you skip one and keep the other? What would happen?

Q10 [Concept · 3 marks]. Why does the signature column update only happen when the row contains a 1? Describe in 2–3 sentences what would go wrong if you updated unconditionally.

Q11 [Concept · 4 marks]. For LSH with b=25,r=4b = 25, r = 4, what is the approximate threshold? At similarity s=0.5s = 0.5, what is the probability of becoming a candidate pair?

Q12 [MCQ · 2 marks]. Which of the following is TRUE about the MinHash signature? (A) It uses the maximum hash value over column entries. (B) Two columns agree on a signature row with probability

equal to their cosine similarity. (C) Two columns agree on a signature row with probability equal to their Jaccard similarity. (D) The signature length is always equal to the number of shingles.

Q13 [MCQ · 2 marks]. Increasing r (rows per band) at fixed $n=brn = b\ r$ does which of the following? (A) Lowers the LSH threshold; more false positives, fewer false negatives. (B) Raises the LSH threshold; fewer false positives, more false negatives. (C) Has no effect on the threshold. (D) Eliminates the need for MinHash.

Q14 [True/False · 2 marks]. "If the Jaccard similarity is 0.9, the LSH banding scheme with $b=20,r=5b = 20,\ r = 5$ will catch the pair as a candidate with probability greater than 99.99%." True or false? Justify with a quick numerical estimate.

Q15 [MCQ · 2 marks]. A document set has $N=106N = 10^6$ documents. Naïve pairwise comparison takes ~50 hours at 10 ns/comparison. With MinHash + LSH at threshold $s=0.8s = 0.8$, what is the dominant computational savings? (A) Reducing each Jaccard from $O(\text{set size})O(\text{set size})$ to $O(\text{signature length})O(\text{signature length})$. (B) Reducing the number of pairs that are explicitly compared from $\sim N2\sim N^2$ to $\sim N\sim N$. (C) Sorting the shingle universe. (D) Better cache locality of hash functions.

---

§12 — Full Worked Answers

A1. Slide 3-shingle window across $abcdabc\texttt{abcdabc}$: $abc\texttt{abc}$, $bcd\texttt{bcd}$, $cda\texttt{cda}$, $dab\texttt{dab}$, $abc\texttt{abc}$. After dedup: $S=\{abc,bcd,cda,dab\}S = \{\texttt{abc}, \texttt{bcd}, \texttt{cda}, \texttt{dab}\}$. There are **4** distinct 3-shingles. (The string $abc\texttt{abc}$ appears at positions 1 and 5 but contributes only once to the set.)

A2. (a) $A\cap B=\{q,s,t\}A \cap B = \{q, s, t\}$, $|A\cap B|=3|A \cap B| = 3$. $A\cup B=\{p,q,r,s,t,u,v\}A \cup B = \{p, q, r, s, t, u, v\}$, $|A\cup B|=7|A \cup B| = 7$. $J(A,B)=3/7\approx 0.429J(A, B) = 3/7 \approx 0.429$. (b) $dJ=1-3/7=4/7\approx 0.571d_J = 1 - 3/7 = 4/7 \approx 0.571$.

A3. Boolean matrix:

aa	1	0	1	0
bb	0	1	1	0
cc	1	1	0	1
dd	0	1	0	1
ee	1	0	1	1

Pairwise Jaccards (compute $|A\cap B||A \cap B|$ and $|A\cup B||A \cup B|$ from the columns):

Pair			
(D1,D2)(D_1, D_2)	{c}\{c\}	{a,b,c,d,e}\{a,b,c,d,e\}	1/5 = 0.20
(D1,D3)(D_1, D_3)	{a,e}\{a, e\}	{a,b,c,e}\{a,b,c,e\}	2/4 = 0.50
(D1,D4)(D_1, D_4)	{c,e}\{c, e\}	{a,c,d,e}\{a,c,d,e\}	2/4 = 0.50
(D2,D3)(D_2, D_3)	{b}\{b\}	{a,b,c,d,e}\{a,b,c,d,e\}	1/5 = 0.20
(D2,D4)(D_2, D_4)	{c,d}\{c, d\}	{b,c,d,e}\{b,c,d,e\}	2/4 = 0.50
(D3,D4)(D_3, D_4)	{e}\{e\}	{a,b,c,d,e}\{a,b,c,d,e\}	1/5 = 0.20

Most-similar pairs (3-way tie): (D1,D3)(D_1, D_3), (D1,D4)(D_1, D_4), (D2,D4)(D_2, D_4) all with J=0.50J = 0.50.

A4. Hashes: h1(x)=(x+2)mod5h_1(x) = (x + 2) \bmod 5. h1(1)=3,h1(2)=4,h1(3)=0,h1(4)=1,h1(5)=2h_1(1)=3, h_1(2)=4, h_1(3)=0, h_1(4)=1, h_1(5)=2. h2(x)=(3x+4)mod5h_2(x) = (3x+4) \bmod 5. h2(1)=2,h2(2)=0,h2(3)=3,h2(4)=1,h2(5)=4h_2(1) = 2, h_2(2) = 0, h_2(3) = 3, h_2(4) = 1, h_2(5) = 4.

Initial M=(∞∞∞∞∞∞∞∞)M = \begin{pmatrix} \infty & \infty & \infty & \infty \\ \infty & \infty & \infty & \infty \end{pmatrix}.

Row 1 (a) boolean (1, 0, 1, 0). h1=3,h2=2h_1=3, h_2=2. Update D1D_1: M[1,D1]=3,M[2,D1]=2M[1,D_1]=3, M[2,D_1]=2. Update D3D_3: M[1,D3]=3,M[2,D3]=2M[1,D_3]=3, M[2,D_3]=2.

h1h_1	3	∞\infty	3	∞\infty
h2h_2	2	∞\infty	2	∞\infty

Row 2 (b) (0, 1, 1, 0). h1=4,h2=0h_1=4, h_2=0. D2D_2: M[1,D2]=4,M[2,D2]=0M[1,D_2]=4, M[2,D_2]=0. D3D_3: M[1,D3]=min(3,4)=3M[1,D_3]=\min(3,4)=3, M[2,D3]=min(2,0)=0M[2,D_3]=\min(2,0)=0.

h1h_1	3	4	3	∞\infty
h2h_2	2	0	0	∞\infty

Row 3 (c) (1, 1, 0, 1). h1=0,h2=3h_1=0, h_2=3. D1D_1: M[1,D1]=min(3,0)=0M[1,D_1]=\min(3,0)=0, M[2,D1]=min(2,3)=2M[2,D_1]=\min(2,3)=2. D2D_2: M[1,D2]=min(4,0)=0M[1,D_2]=\min(4,0)=0, M[2,D2]=min(0,3)=0M[2,D_2]=\min(0,3)=0. D4D_4: M[1,D4]=0,M[2,D4]=3M[1,D_4]=0, M[2,D_4]=3.

h1h_1	0	0	3	0
h2h_2	2	0	0	3

Row 4 (d) (0, 1, 0, 1). h1=1,h2=1h_1=1, h_2=1. D2D_2: M[1,D2]=min(0,1)=0M[1,D_2]=\min(0,1)=0, M[2,D2]=min(0,1)=0M[2,D_2]=\min(0,1)=0. D4D_4: M[1,D4]=min(0,1)=0M[1,D_4]=\min(0,1)=0, M[2,D4]=min(3,1)=1M[2,D_4]=\min(3,1)=1.

h1h_1	0	0	3	0
h2h_2	2	0	0	1

Row 5 (e) (1, 0, 1, 1). h1=2,h2=4h_1=2, h_2=4. D1D_1: M[1,D1]=min(0,2)=0M[1,D_1]=\min(0,2)=0, M[2,D1]=min(2,4)=2M[2,D_1]=\min(2,4)=2. D3D_3: M[1,D3]=min(3,2)=2M[1,D_3]=\min(3,2)=2, M[2,D3]=min(0,4)=0M[2,D_3]=\min(0,4)=0. D4D_4: M[1,D4]=min(0,2)=0M[1,D_4]=\min(0,2)=0, M[2,D4]=min(1,4)=1M[2,D_4]=\min(1,4)=1.

Final signature matrix:

h1h_1	0	0	2	0
h2h_2	2	0	0	1

Estimated Jaccards: Ĵ(D1,D2)=1/2=0.50\hat{J}(D_1, D_2) = 1/2 = 0.50 (agree on h1h_1). True 0.20. Ĵ(D1,D4)=1/2\hat{J}(D_1, D_4) = 1/2 (agree on h1h_1). True 0.50 — close. Ĵ(D2,D4)=1/2\hat{J}(D_2, D_4) = 1/2 (agree on h1h_1). True 0.50 — match. With only 2 hashes, estimates are noisy.

A5. (a) Ĵ=78/100=0.78\hat{J} = 78/100 = 0.78. (b) Var =0.78·0.22/100=0.001716= 0.78 \cdot 0.22 / 100 = 0.001716. SE =0.001716≈0.0414= \sqrt{0.001716} \approx 0.0414. (c) 95% CI: 0.78±1.96·0.0414=0.78±0.081=[0.699,0.861]0.78 \pm 1.96 \cdot 0.0414 = 0.78 \pm 0.081 = [0.699, 0.861]. So the true JJ is plausibly anywhere from ~0.70 to ~0.86.

A6. Bands of 2 rows.

- **Band 1 (rows 1–2):** C1=(1,4)C_1 = (1,4), C2=(2,5)C_2 = (2,5), C3=(1,4)C_3 = (1,4), C4=(3,6)C_4 = (3,6). Collision: (C1,C3)(C_1, C_3).
- **Band 2 (rows 3–4):** C1=(7,1)C_1 = (7,1), C2=(7,1)C_2 = (7,1), C3=(8,2)C_3 = (8,2), C4=(9,3)C_4 = (9,3). Collision: (C1,C2)(C_1, C_2).
- **Band 3 (rows 5–6):** C1=(9,0)C_1 = (9,0), C2=(9,0)C_2 = (9,0), C3=(9,1)C_3 = (9,1), C4=(4,5)C_4 = (4,5). Collision: (C1,C2)(C_1, C_2).
- **Band 4 (rows 7–8):** C1=(3,2)C_1 = (3,2), C2=(6,4)C_2 = (6,4), C3=(3,2)C_3 = (3,2), C4=(6,4)C_4 = (6,4). Collisions: (C1,C3)(C_1, C_3) and (C2,C4)(C_2, C_4).

Candidate pairs (union over bands): {(C1,C2),(C1,C3),(C2,C4)}\{(C_1, C_2), (C_1, C_3), (C_2, C_4)\}.

(Note: (C3,C4)(C_3, C_4) never collide → excluded. (C2,C3)(C_2, C_3) never collide → excluded.)

A7. Theorem: Prπ[hπ(C1)=hπ(C2)]=J(C1,C2)\Pr_\pi[h_\pi(C_1) = h_\pi(C_2)] = J(C_1, C_2). Classify rows by their pattern in columns C1,C2C_1, C_2: Type A = (1,1), Type B = (1,0), Type C = (0,1), Type D = (0,0). Under a random row permutation, the *first* row (in permuted order) that is *not type D* is uniformly random among the a+b+ca + b + c non-D rows. The two columns minhash to the same value if and only if that first non-D row is type A, which happens with probability a/(a+b+c)a / (a+b+c). But a/(a+b+c)=|C1∩C2|/|C1∪C2|=J(C1,C2)a / (a+b+c) = |C_1 \cap C_2| / |C_1 \cup C_2| = J(C_1, C_2).

A8. With k=2k = 2, the universe is at most 262=67626^2 = 676 character bigrams. English text has skewed bigram frequencies (e.g. “th”, “he”, “in” are everywhere), so any two random English documents share most of their bigrams. **Example:** “the cat sat” and “the dog ran” share bigrams “th”, “he” (and possibly “e”, “ c”/” d”, “ s”/” r”). Two unrelated paragraphs about totally different topics may still hit Jaccard ≈ 0.4–0.6 just from shared common letters. With k=5k = 5, the universe explodes to 265≈10726^5 \approx 10^7 and unrelated documents share very few 5-shingles.

A9. MinHash turns each huge shingle set into a small fixed-length integer signature, *while approximately preserving* Jaccard similarity. **LSH banding** turns the O(N2)O(N^2) all-pairs comparison into a fast bucketing step that produces only a small list of candidate pairs.

If you skip MinHash but keep banding: you need to band over the original boolean matrix, which is huge — banding rows of length ~10710^7 instead of length ~100. Very slow, doesn’t compress.

If you skip banding but keep MinHash: you have small signatures but still must compare every pair of them — still O(N2)O(N^2) comparisons. Each is faster but the count is unchanged.

You need both. MinHash compresses; LSH avoids enumerating non-candidate pairs.

A10. MinHash is defined as the **first row in which the column has a 1**. If you update for every row (regardless of whether the column has a 1), you compute minrhi(r)\min_r h_i(r) over *all* rows — a quantity independent of the column. Every column’s signature would then equal the same constant, carrying zero information about the column’s content. The whole point of conditioning on “column has a 1” is to make the signature a function of the *column’s set*, not just the universe.

A11. Threshold t≈(1/b)1/r=(1/25)1/4=0.040.25t \approx (1/b)^{1/r} = (1/25)^{1/4} = 0.04^{0.25}. log(0.04)=−1.398\log(0.04) = -1.398, −1.398/4=−0.350-1.398 / 4 = -0.350, 10−0.350≈0.44710^{-0.350} \approx 0.447. So **threshold ≈ 0.45**.

At s=0.5s = 0.5: sr=0.54=0.0625s^r = 0.5^4 = 0.0625. 1−0.0625=0.93751 - 0.0625 = 0.9375. 0.937525≈0.2020.9375^{25} \approx 0.202. Pr[candidate]=1−0.202=0.798\Pr[\text{candidate}] = 1 - 0.202 = 0.798 ≈ **80%**.

A12. (C). That is the MinHash theorem. - (A) wrong: minimum, not maximum. - (B) wrong: Jaccard, not cosine. - (D) wrong: signature length is set by user (~100), not by the universe.

A13. (B). Larger rr means each band requires more rows to all match → less likely to collide → harder to be a candidate → threshold rises → fewer false positives but more false negatives.

A14. True. 1−(1−0.95)20=1−(1−0.59049)20=1−0.40951201 - (1 - 0.9^5)^{20} = 1 - (1 - 0.59049)^{20} = 1 - 0.40951^{20}. 0.4095120≈exp(20ln0.40951)=exp(20·(−0.893))=exp(−17.86)≈1.7×10−80.40951^{20} \approx \exp(20 \ln 0.40951) = \exp(20 \cdot (-0.893)) = \exp(-17.86) \approx 1.7 \times 10^{-8}. So

$\Pr = 1 - 1.7 \times 10^{-8} \approx 0.99999998$ — vastly greater than 99.99%. **True.**

A15. (B). The headline saving is the LSH banding step, which prunes the candidate set from $\binom{N}{2} \approx 5 \times 10^{11}$ down to the small subset of pairs that share a band — typically of order N. (A) is true but secondary: shrinking each comparison by a factor of ~1000 saves a constant factor, but the asymptotic win is from cutting N^2 to $\approx N$.

---

§13 — Ending Key Notes (Revision Cards)

Term	Quick-fact
3-step pipeline	Shingling → MinHash → LSH. Documents → sets → small signatures → candidate pairs.
k-shingle	Length-kk contiguous substring (or word kk-gram). Document → set of kk-shingles.
Choice of kk	k=5k = 5 short docs, k=8k = 8–1010 long docs. Too small → false matches.
Hashing shingles	Map each kk-shingle to 4 bytes via a hash. Sets become sets of integers.
Boolean matrix	Rows = shingles, cols = documents. Entry = 1 iff shingle in doc. Sparse.
Jaccard similarity	J(A,B)=∣∣A∩B∣∣/∣∣A∪B∣∣J(A, B) = |A \cap B| / |A \cup B|. Symmetric, range [0,1][0, 1].
Jaccard distance	dJ=1−Jd_J = 1 - J. A true metric.
Type-A/B/C/D rows	For pair (C1,C2)(C_1, C_2): classify each row by (1,1)/(1,0)/(0,1)/(0,0). J=a/(a+b+c)J = a/(a+b+c).
MinHash hπ(C)h_\pi(C)	Index of the first 1 in column CC under permutation π\pi.
MinHash theorem	Pr[hπ(A)=hπ(B)]=J(A,B)\Pr[h_\pi(A) = h_\pi(B)] = J(A, B).
Signature matrix	Rows = hash functions, cols = documents. Each entry an integer.
Estimating JJ from sigs	J^\hat{J} = fraction of signature rows on which two columns agree.
Hash family for permutations	hi(x)=(aix+bi)modpmodMh_i(x) = (a_i x + b_i) \bmod p \bmod M. ∼100\sim 100 such hashes simulate permutations.
MinHash algorithm	Init M[i,c]=∞M[i,c] = \infty. For each row rr, update M[i,c]M[i,c] only for columns with 1 in rr.
LSH banding	Split sig into bb bands of rr rows. Hash each band's slice into a bucket.
Candidate pair	Pair of docs sharing a bucket in ≥1\geq 1 band.
S-curve	Pr[cand∣s]=1−(1−sr)b\Pr[\text{cand} \mid s] = 1 - (1 - s^r)^b. Sigmoid.
Threshold formula	t≈(1/b)1/rt \approx (1/b)^{1/r}. Pairs above tt likely caught; below tt likely filtered.
FP / FN trade-off	Increase rr → threshold up → fewer FPs, more FNs. Increase bb → threshold down.
Variance of J^\hat{J}	∼J(1−J)/n\sim J(1-J)/n. SE ≈1/n\approx 1/\sqrt{n}. n=100⇒n=100 \Rightarrow SE ≈0.10\approx 0.10.
SimHash / cosine LSH	Random hyperplanes. Pr[same bit]=1−θ/π\Pr[\text{same bit}] = 1 - \theta/\pi.

Term	Quick-fact
Mistake — MIN not max	MinHash uses **min**. Calling it MaxHash gives garbage.
Mistake — bands vs rows	bb = bands, rr = rows per band. n=brn = b r.
Mistake — update only on 1	Only update M[i,c]M[i,c] if column cc has a 1 in the current row. Else skip.
Mistake — threshold formula	t≈(1/b)1/rt \approx (1/b)^{1/r}, NOT 1/(br)1/(br) and NOT 1/(b+r)1/(b+r).

§14 — Formula & Algorithm Reference

Concept	Formula	When to use
k-shingle set	$S_k(D) = \{c_i \ldots c_{i+k-1} : 1 \leq i \leq L-k+1\}$	Step 1 of pipeline. Convert document to set.
Jaccard similarity	$J(A,B) = \|A \cap B\| / \|A \cup B\|$	Default similarity for sets.
Jaccard from row types	$J = a/(a+b+c)$, types A/B/C of rows	Computing J from a boolean matrix.
MinHash signature	$h_\pi(C) = \min \{ \pi(r) : C[r] = 1 \}$	Step 2. Compress a set to one integer.
MinHash theorem	$\Pr[h_\pi(A) = h_\pi(B)] = J(A, B)$	Justifies why MinHash preserves similarity.
Hash family for permutations	$h_i(x) = (a_i x + b_i) \bmod p \bmod M$	Replace permutation by hash; one disk pass.
Estimator from signatures	$\hat{J} = \frac{1}{n}\sum_i \mathbf{1}[M[i, C_1] = M[i, C_2]]$	Estimate Jaccard at query time.
Variance of estimator	$\text{Var}(\hat{J}) = J(1-J)/n$	Predict precision of signature-based estimate.
Banding probability	$\Pr[\text{candidate} \mid s] = 1 - (1 - s^r)^b$	LSH analysis; S-curve.
Banding threshold	$t \approx (1/b)^{1/r}$	Tune b, r for desired similarity threshold.
Total signature length	$n = b \cdot r$	Constraint when allocating signature size.

Algorithmic pseudocode (memorize)

MinHash signature construction:

```
 for i = 1..n: for c = 1..N: M[i, c] = ∞
for r = 1, 2, ..., R (rows):
    for i = 1..n: v_i = h_i(r)
    for c = 1..N where C[r] = 1:
        for i = 1..n:
            M[i, c] = min(M[i, c], v_i)
```

LSH banding:

```
for band b = 1..B:
    for column c = 1..N:
        slice = M[(b-1)*r+1 .. b*r, c]
        bucket = hash(slice)
        push (band b, bucket) → c into bucket-table
for each (band, bucket) with ≥ 2 columns:
    every pair of those columns is a candidate
candidates = ∪ over all bands
```

Complexities

Step	Time	Space
Shingling	$O(L)$ per document	$O(L)$ per document
MinHash sigs	$O(R \cdot n)$ amortized over all docs	$O(N \cdot n)$ for sig matrix
LSH banding	$O(N \cdot b)$ hash lookups	$O(N \cdot b)$ buckets
Candidate verify	$O(\#candidates \cdot n)$	$O(1)$ per check

Connections to other weeks

- **W6–W7 — Frequent itemsets / Apriori:** also use the "find similar items" lens, but for transactions.
- **W12 — TrustRank, Topic-Sensitive PageRank:** different LSH-style filtering for biased random walks.
- **W14 — Recommendation systems:** MinHash on user-item sets for collaborative filtering.
- **MMDS Chapter 3:** the canonical reference. Sections 3.1–3.4 are the syllabus core; 3.5–3.7 cover other distance LSH families (cosine, Euclidean, Hamming).

End of W04-05 Finding Similar Items exam-prep document.